Chapter 16: Cluster Analysis

> (c) 2025 Galit Shmueli, Peter C. Bruce, Peter Gedeck
>
> Code included in
>
> *Machine Learning for Business Analytics: Concepts, Techniques, and Applications in Python* (Second Edition) Galit Shmueli, Peter C. Bruce, Peter Gedeck, and Nitin R. Patel. 2025.

Add comments to each **code** cell (no comments need to be added for Markdown).

Each code cell starts with two comments:

1. Your initials and last two digits of your ID number.
2. Provide a brief explanation of the code cell. If comments are already included in the code cell, you may revise them for clarity, summarize the existing comments, or leave them unchanged if they are already appropriate. For example,

JT88

This code cell imports necessary Python packages.

In [1]:
```
pip install adjustText
```

```
Requirement already satisfied: adjustText in /opt/anaconda3/lib/python3.13/s
ite-packages (1.3.0)
Requirement already satisfied: numpy in /opt/anaconda3/lib/python3.13/site-p
ackages (from adjustText) (2.1.3)
Requirement already satisfied: matplotlib in /opt/anaconda3/lib/python3.13/s
ite-packages (from adjustText) (3.10.0)
Requirement already satisfied: scipy in /opt/anaconda3/lib/python3.13/site-p
ackages (from adjustText) (1.15.3)
Requirement already satisfied: contourpy>=1.0.1 in /opt/anaconda3/lib/python
3.13/site-packages (from matplotlib->adjustText) (1.3.1)
Requirement already satisfied: cycler>=0.10 in /opt/anaconda3/lib/python3.1
3/site-packages (from matplotlib->adjustText) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in /opt/anaconda3/lib/pytho
n3.13/site-packages (from matplotlib->adjustText) (4.55.3)
Requirement already satisfied: kiwisolver>=1.3.1 in /opt/anaconda3/lib/pytho
n3.13/site-packages (from matplotlib->adjustText) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /opt/anaconda3/lib/python
3.13/site-packages (from matplotlib->adjustText) (24.2)
Requirement already satisfied: pillow>=8 in /opt/anaconda3/lib/python3.13/si
te-packages (from matplotlib->adjustText) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in /opt/anaconda3/lib/python
3.13/site-packages (from matplotlib->adjustText) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in /opt/anaconda3/lib/py
thon3.13/site-packages (from matplotlib->adjustText) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /opt/anaconda3/lib/python3.13/sit
e-packages (from python-dateutil>=2.7->matplotlib->adjustText) (1.17.0)
Note: you may need to restart the kernel to use updated packages.
```

In [2]:

```
import matplotlib.pyplot as plt
import mlba
import pandas as pd
import seaborn as sns # advanced plotting
from adjustText import adjust_text # prevent overlapping lables in plots
from pandas.plotting import parallel_coordinates
from scipy.cluster.hierarchy import dendrogram, fcluster, linkage
from sklearn import preprocessing
from sklearn.cluster import KMeans
from sklearn.metrics import pairwise # pairwise distances for similarity cal
%matplotlib inline
```

```
no display found. Using non-interactive Agg backend
```

Example: Public Utilities

Table 16.1 gives corporate data on 22 public utilities in the United States (the variable definitions are given in the table footnote). We are interested in forming groups of similar utilities. The records to be clustered are the utilities, and the clustering will be based on the eight measurements on each utility. An example where clustering would be useful is a study to predict the cost impact of deregulation. To do the requisite analysis, economists would need to build a detailed cost model of the various utilities. It would save a considerable amount of time and effort if we could cluster similar types of utilities and build detailed cost models for just one “typical” utility in each cluster and then scale up from these models to estimate results for all utilities.

For simplicity, let us consider only two of the measurements: Sales and Fuel Cost. Figure 16.1 shows a scatter plot of these two variables, with labels marking each company. At first glance, there appear to be two or three clusters of utilities: one with utilities that have high fuel costs, a second with utilities that have lower fuel costs and relatively low sales, and a third with utilities with low fuel costs but high sales.

Figure 16.1 Scatter Plot of Fuel Cost vs. Sales for the 22 Utilities

In [3]:
```
#VB67
# Loading the Utilities dataset
utilities_df = mlba.load_data('Utilities.csv')
# Creating a scatter plot of Sales vs. Fuel Cost
ax = utilities_df.plot.scatter(x='Sales', y='Fuel_Cost')
# Adding company names as text labels for each data point on the plot
texts = []
for _, row in utilities_df.iterrows():
    texts.append(ax.text(row.Sales, row.Fuel_Cost, row.Company))

# Adjusting text labels to avoid overlapping, adding light arrows for clarit
adjust_text(texts, arrowprops={'arrowstyle': '-|>', 'alpha': 0.5})
# Label the x-axis and y-axis
ax.set_xlabel('Sales')
ax.set_ylabel('Fuel cost')
# Adjusting layout for better spacing and show the final plot
plt.tight_layout()
#displaying plot
plt.show()
```

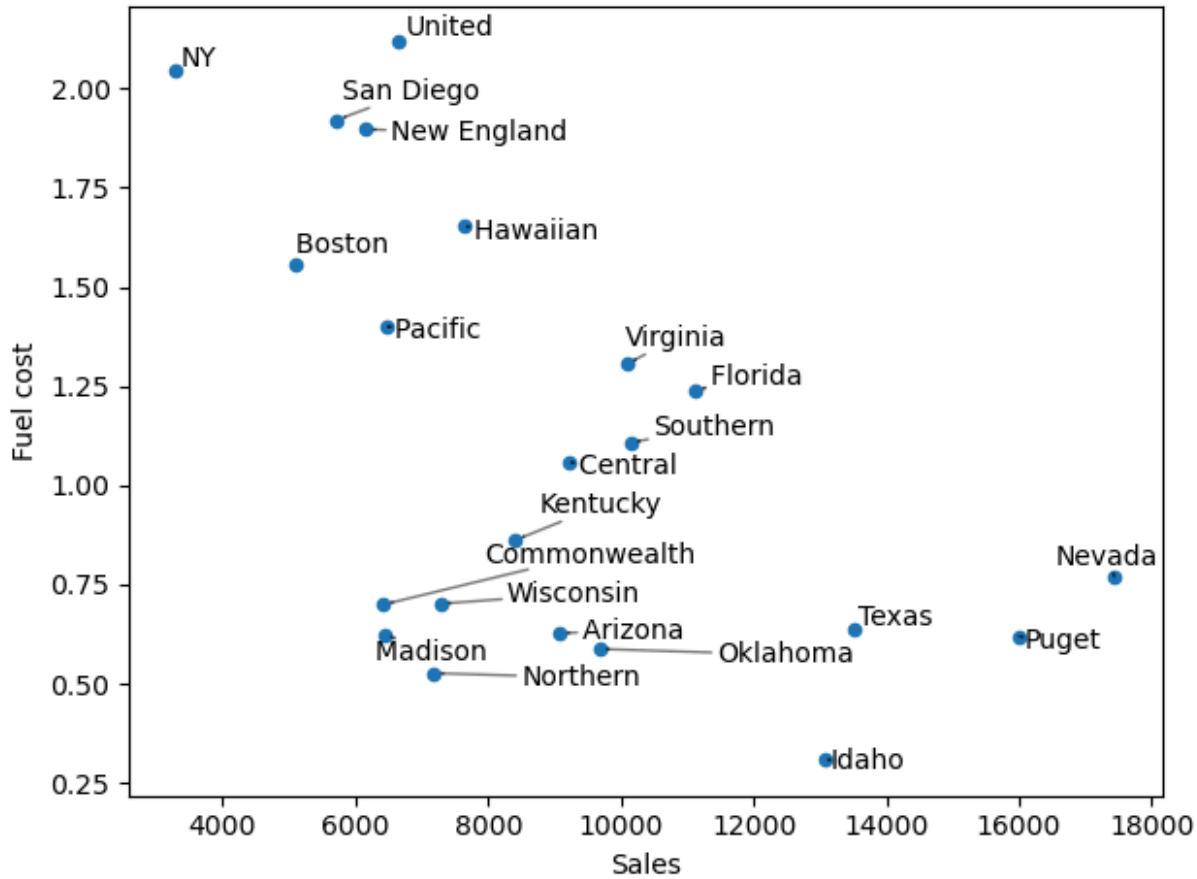


Table 16.2 Distance Matrix Between Pairs of Utilities, Using Euclidean Distance

In [4]:
```
#VB67
# Loading the Utilities dataset
utilities_df = mlba.load_data('Utilities.csv')

# setting row names to the utilities column
utilities_df = utilities_df.set_index('Company')

# computing Euclidean distance
d = pairwise.pairwise_distances(utilities_df, metric='euclidean')
pd.DataFrame(d, columns=utilities_df.index, index=utilities_df.index)
```

Out[4]:

Company	Arizona	Boston	Central	Commonwealth	
Company					
Arizona	0.000000	3989.408076	140.402855	2654.277632	5777.1676
Boston	3989.408076	0.000000	4125.044132	1335.466502	1788.0680
Central	140.402855	4125.044132	0.000000	2789.759674	5912.5529
Commonwealth	2654.277632	1335.466502	2789.759674	0.000000	3123.1532
NY	5777.167672	1788.068027	5912.552908	3123.153215	0.0000
Florida	2050.529440	6039.689076	1915.155154	4704.363099	7827.429
Hawaiian	1435.265019	2554.287162	1571.295401	1219.560005	4342.0937
Idaho	4006.104187	7994.155985	3872.257626	6659.534567	9782.158
Kentucky	671.276346	3318.276558	807.920792	1983.314354	5106.0941
Madison	2622.699002	1367.090634	2758.559663	43.648894	3155.0955
Nevada	8364.031051	12353.062698	8229.223281	11018.057812	14141.0225
New England	2923.136103	1066.579432	3058.707429	271.452731	2854.0994
Northern	1899.279821	2091.160485	2035.441520	756.831954	3879.1674
Oklahoma	598.556633	4586.302564	461.341670	3250.984589	6373.7432
Pacific	2609.045363	1380.749962	2744.502847	56.644626	3168.1774
Puget	6914.742065	10903.146464	6780.430307	9568.434429	12691.1551
San Diego	3363.061626	629.760748	3498.113013	710.292965	2414.6987
Southern	1063.009074	5052.331669	928.749249	3717.202963	6840.1502
Texas	4430.251585	8419.610541	4295.014690	7084.372839	10207.3926
Wisconsin	1790.485648	2199.721665	1925.772564	864.273153	3987.3359
United	2427.588875	1562.210811	2563.637362	232.476871	3350.073
Virginia	1016.617691	5005.081262	883.535455	3670.018191	6793.0353

22 rows × 22 columns

Normalizing Numerical Variables

The measure computed above is highly influenced by the scale of each variable so that variables with larger scales (e.g., Sales) have a much greater influence over the total distance. It is therefore customary to normalize continuous measurements before computing the Euclidean distance. This converts all measurements to the same scale. Normalizing a measurement means subtracting the average and dividing by the standard

deviation (normalized values are also called z-scores). For instance, the average sales amount across the 22 utilities is 8914.045, and the standard deviation is 3549.984.

Returning to the simplified utilities data with only two measurements (Sales and Fuel Cost), we first normalize the measurements, and then compute the Euclidean distance between each pair.

Table 16.4 Distance Matrix Between Pairs of Utilities, Using Euclidean Distance and Normalized Variables

In [5]:
```
#VB67
# scikit-learn uses population standard deviation
utilities_df_norm = utilities_df.apply(preprocessing.scale, axis=0) # approa

# pandas uses sample standard deviation
utilities_df_norm = (utilities_df - utilities_df.mean())/utilities_df.std()

# computing normalized distance based on Sales and Fuel Cost
utilities_df_norm[['Sales', 'Fuel_Cost']]
d_norm = pairwise.pairwise_distances(utilities_df_norm[['Sales', 'Fuel_Cost'
                                     metric='euclidean')
pd.DataFrame(d_norm, columns=utilities_df.index, index=utilities_df.index)
```

Out[5]:

Company	Arizona	Boston	Central	Commonwealth	NY	Florida
Company						
Arizona	0.000000	2.010329	0.774179	0.758738	3.021907	1.244422
Boston	2.010329	0.000000	1.465703	1.582821	1.013370	1.792397
Central	0.774179	1.465703	0.000000	1.015710	2.432528	0.631892
Commonwealth	0.758738	1.582821	1.015710	0.000000	2.571969	1.643857
NY	3.021907	1.013370	2.432528	2.571969	0.000000	2.635573
Florida	1.244422	1.792397	0.631892	1.643857	2.635573	0.000000
Hawaiian	1.885248	0.740283	1.156092	1.746027	1.411695	1.228805
Idaho	1.265638	3.176654	1.732777	2.003230	4.162561	1.764123
Kentucky	0.461292	1.557738	0.419254	0.629994	2.566439	1.025663
Madison	0.738650	1.719632	1.102287	0.138758	2.705445	1.722510
Nevada	2.369479	3.756513	2.375975	3.106084	4.597006	1.971518
New England	2.425975	0.684393	1.737322	2.153831	0.846291	1.831380
Northern	0.564657	1.940166	1.113433	0.377004	2.938637	1.698624
Oklahoma	0.182648	2.166078	0.855093	0.937389	3.174588	1.243634
Pacific	1.570780	0.478334	0.987772	1.258835	1.462019	1.343185
Puget	1.947668	3.501390	2.065643	2.699060	4.397433	1.767581
San Diego	2.509043	0.679634	1.836762	2.202930	0.715629	1.953423
Southern	0.913621	1.634425	0.276440	1.278514	2.558409	0.366744
Texas	1.247976	2.890560	1.428159	1.998818	3.831132	1.277920
Wisconsin	0.521491	1.654255	0.838967	0.243408	2.661786	1.452417
United	2.761745	1.100595	2.034824	2.547116	0.952507	2.016493
Virginia	1.252350	1.479261	0.510365	1.502093	2.328691	0.313847

22 rows × 22 columns

Partial output

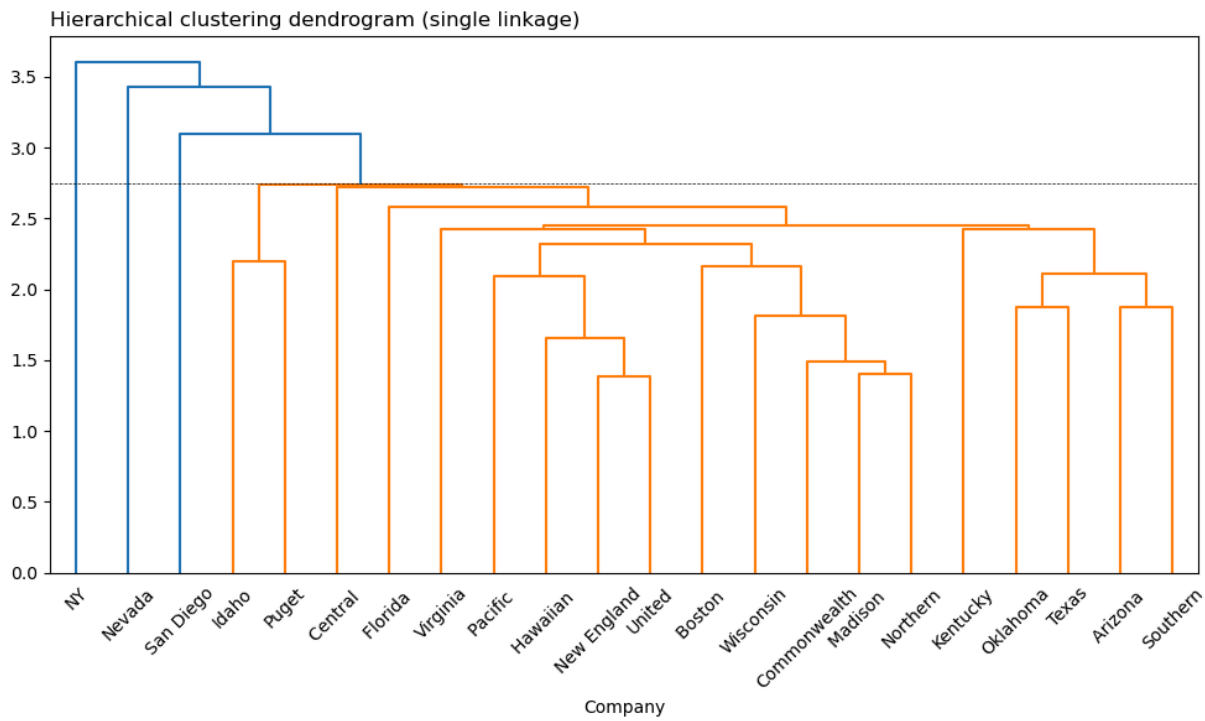
HIERARCHICAL (AGGLOMERATIVE) CLUSTERING

Dendrograms: Displaying Clustering Process and Results

A dendrogram is a treelike diagram that summarizes the process of clustering. On the x-axis are the records. Similar records are joined by lines whose vertical length reflects the distance between the records. Figure 16.3 shows the dendrograms that results from clustering the 22 utilities using the 8 normalized variables, Euclidean distance, once with single linkage (top), and once with average linkage (bottom).

In [6]:
```
#VB67
def plot_dendrogram(data, method, color_threshold):
    # in linkage() set argument method =
    # 'single', 'complete', 'average', 'weighted', centroid', 'median', 'war
    Z = linkage(data, method=method)
    # Creating a figure and axis for plotting
    fig, ax = plt.subplots(figsize=(10, 6))
    # Setting the title and x-axis label for the dendrogram
    ax.set_title(f'Hierarchical clustering dendrogram ({method} linkage)', l
    ax.set_xlabel('Company')
    # Plotting the dendrogram using the computed linkage matrix
    dendrogram(Z, labels=data.index, color_threshold=color_threshold, ax=ax)
    ax.axhline(y=color_threshold, color='black', linewidth=0.5, linestyle='c
```

In [7]:
```
#VB67
plot_dendrogram(utilities_df_norm, 'single', 2.75) # threshold = 2.75 shows
plt.tight_layout()
#displaying plot
plt.show()
```



In [8]:
```
#VB67
plot_dendrogram(utilities_df_norm, 'average', 3.6) # threshold = 3.6 shows p
plt.tight_layout()
#displaying plot
plt.show()
```

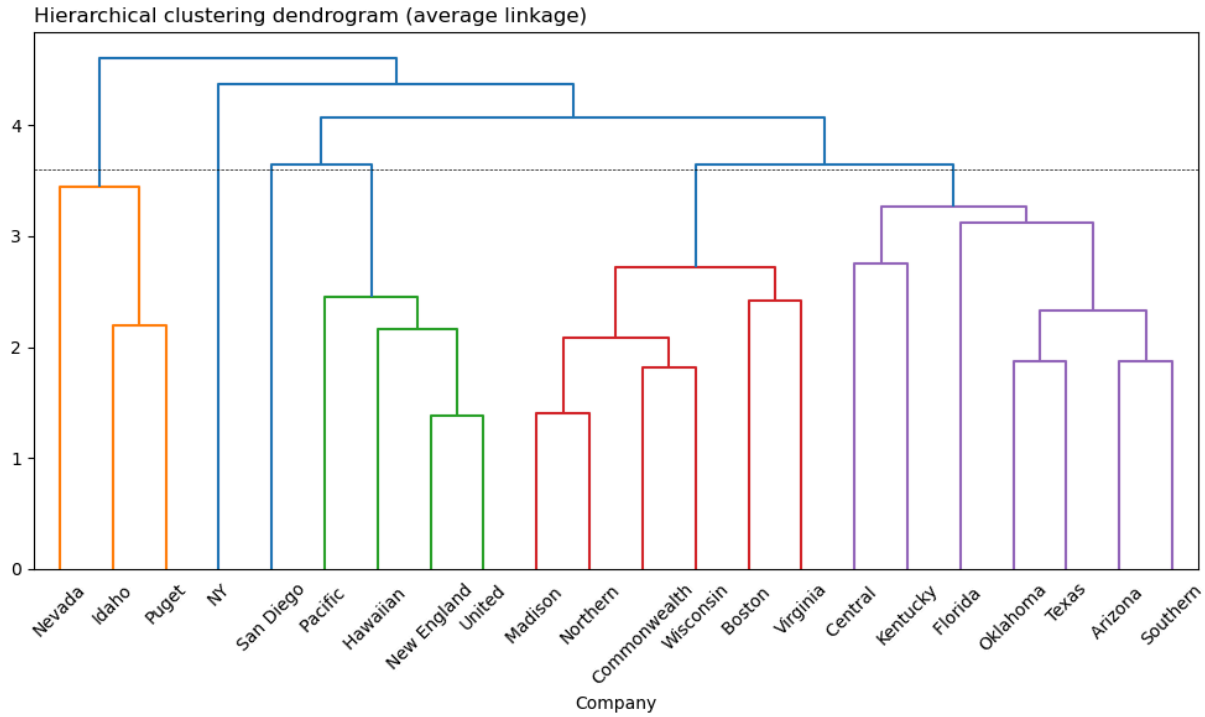


Table 16.6 Computing Cluster Membership By "Cutting" the Endrogram

Clusters can be computed with "scipy" by applying the function fcluster() to the dendrogram object. Table 16.6 shows the results for the single linkage and the average linkage clustering with six clusters. Each record is assigned a cluster number. While some records remain in the same cluster in both methods (e.g., Arizona, Florida, Kentucky, Oklahoma, Texas), others change.

Single Linkage (output modified for clarity)

In [9]:
```
#VB67
memb = fcluster(linkage(utilities_df_norm, method='single'), 6, criterion='m
memb = pd.Series(memb, index=utilities_df_norm.index) # create a series "mem
for key, item in memb.groupby(memb):
    print(key, ': ', ', '.join(item.index))
```

```
1 :  Idaho, Puget
2 :  Arizona , Boston , Commonwealth, Florida , Hawaiian , Kentucky, Madison
, New England, Northern, Oklahoma, Pacific , Southern, Texas, Wisconsin, Uni
ted, Virginia
3 :  Central
4 :  San Diego
5 :  Nevada
6 :  NY
```

Average Linkage (output modified for clarity)

In [10]:
```
#VB67
# Performing hierarchical clustering using the 'average' linkage method and
```

```
memb = fcluster(linkage(utilities_df_norm, method='average'), 6, criterion='
# Converting the cluster assignments into a pandas Series with company names
memb = pd.Series(memb, index=utilities_df_norm.index)
# Printing each cluster number along with the list of companies belonging to
for key, item in memb.groupby(memb):
    print(key, ': ', ', '.join(item.index))
```

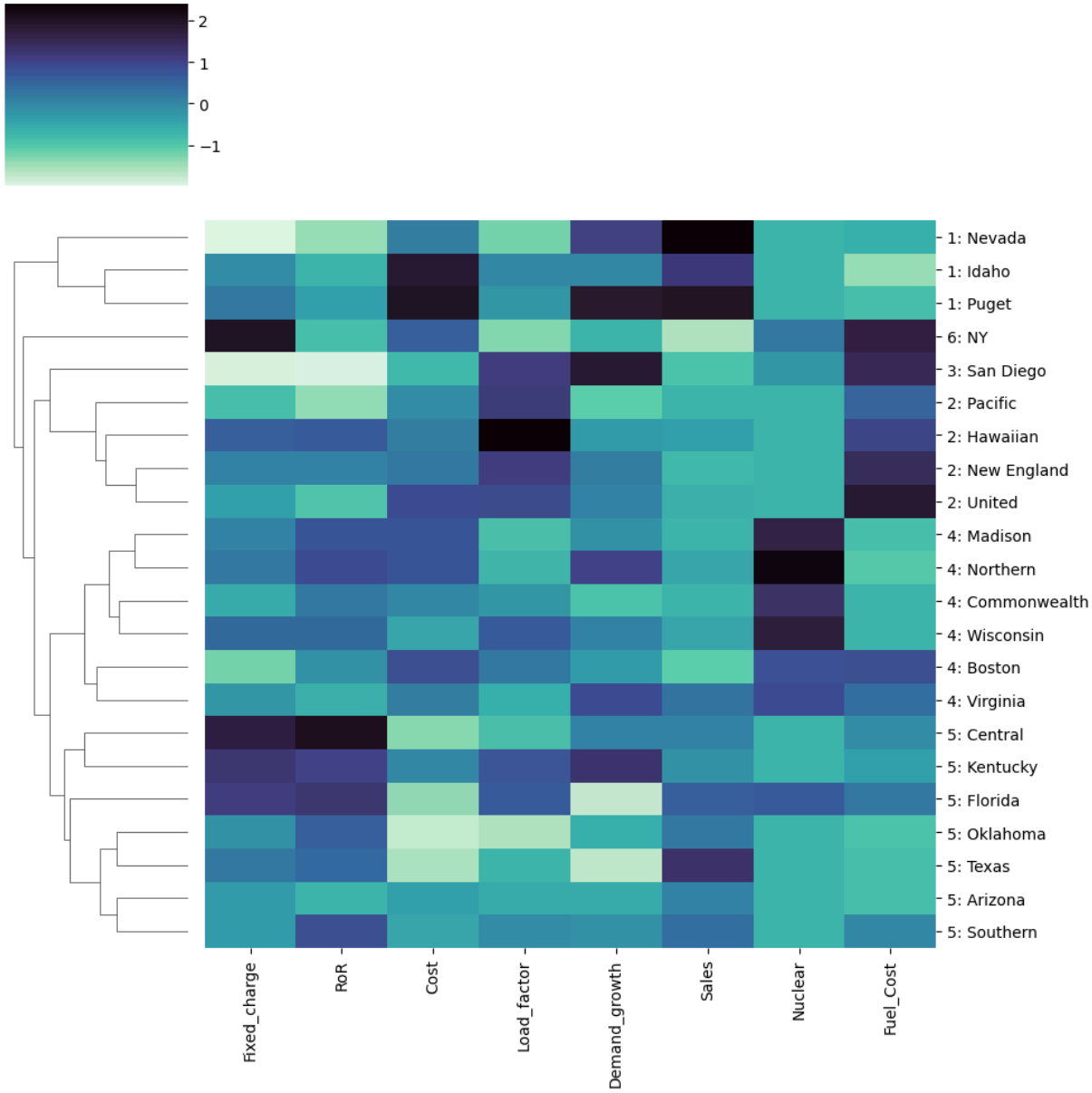
```
1 :  Idaho, Nevada, Puget
2 :  Hawaiian , New England, Pacific , United
3 :  San Diego
4 :  Boston , Commonwealth, Madison , Northern, Wisconsin, Virginia
5 :  Arizona , Central , Florida , Kentucky, Oklahoma, Southern, Texas
6 :  NY
```

Figure 16.4 Heatmap for the 22 Utilities.

Rows are sorted by the six clusters from average linkage clustering. Darker cells denote higher values within a column.

We can further characterize each of the clusters by examining the summary statistics of their measurements, or visually looking at a heatmap of their individual measurements. Figure 16.4 shows a heatmap of the four clusters and two singletons, highlighting the different profile that each cluster has in terms of the eight measurements. We see, for instance, that cluster 4 is characterized by utilities with a high percent of nuclear power; cluster 5 is characterized by high fixed charge and RoR; cluster 2 has high fuel costs.

In [11]:
```
#VB67
# setting labels as cluster membership and utility name
utilities_df_norm.index = [f'{cluster}: {state}'
                           for cluster, state in zip(memb, utilities_df_norm
# plotting heatmap
# the '_r' suffix reverses the color mapping to large = dark
sns.clustermap(utilities_df_norm, method='average', col_cluster=False, cmap=
plt.tight_layout()
#displaying plot
plt.show()
```



NONHIERARCHICAL CLUSTERING: THE k-MEANS ALGORITHM

Table 16.9 k-Means Clustering of 22 Utilities into k = 6 Clusters

The results of running the k-means algorithm using class KMeans() from scikit-learn for all 22 utilities and eight measurements with k =6 are shown in Table 16.9.

In [12]:
```
#VB67
# Loading and preprocess data
utilities_df = mlba.load_data('Utilities.csv')
utilities_df = utilities_df.set_index('Company')
utilities_df = utilities_df.apply(lambda x: x.astype('float64'))
```

```
# Normalize distances
utilities_df_norm = utilities_df.apply(preprocessing.scale, axis=0)

kmeans = KMeans(n_clusters=6, random_state=0).fit(utilities_df_norm) # runs

# Cluster membership
memb = pd.Series(kmeans.labels_, index=utilities_df_norm.index)
for key, item in memb.groupby(memb):
    print(key, ': ', ', '.join(item.index))
```

```
0 :  Commonwealth, Madison , Northern, Wisconsin, Virginia
1 :  Pacific
2 :  Arizona , Central , Florida , Oklahoma, Southern, Texas
3 :  Idaho, Nevada, Puget
4 :  Kentucky
5 :  Boston , NY, Hawaiian , New England, San Diego, United
```

Table 16.10 Cluster Centroids and Squared Distances for k-means with k = 6

Centroids

In [13]:
```
#VB67
# Creating a DataFrame of k-means cluster centroids using normalized variabl
centroids = pd.DataFrame(kmeans.cluster_centers_, columns=utilities_df_norm.
# Setting the display precision to 3 decimal places for cleaner output
pd.set_option('display.precision', 3)
# Displaying the cluster centroids
centroids
```

Out[13]:

	Fixed_charge	RoR	Cost	Load_factor	Demand_growth	Sales	Nuclear	Fuel_
0	-0.012	0.339	0.224	-0.366	0.170	-0.411	1.602	-(
1	-0.855	-1.431	-0.104	1.198	-1.097	-0.705	-0.677	(
2	0.393	0.759	-1.177	-0.534	-0.801	0.445	-0.503	-(
3	-0.614	-0.853	1.370	-0.492	1.015	1.900	-0.731	-(
4	1.253	1.032	-0.005	0.785	1.300	-0.146	-0.731	-(
5	-0.143	-0.549	0.323	0.755	0.118	-0.910	-0.232	

Within-cluster sum of squared distances and cluster count

In [14]:
```
#VB67
# calculating the distances of each data point to the cluster centers
distances = kmeans.transform(utilities_df_norm)

# finding closest cluster for each data point
minSquaredDistances = distances.min(axis=1) ** 2

# combining with cluster labels into a data frame
df = pd.DataFrame({'squaredDistance': minSquaredDistances, 'cluster': kmeans
```

```
    index=utilities_df_norm.index)

# grouping by cluster and print information
for cluster, data in df.groupby('cluster'):
    count = len(data)
    withinClustSS = data.squaredDistance.sum()
    print(f'Cluster {cluster} ({count} members): {withinClustSS:.2f} within
```

```
Cluster 0 (5 members): 10.66 within cluster
Cluster 1 (1 members): 0.00 within cluster
Cluster 2 (6 members): 20.28 within cluster
Cluster 3 (3 members): 9.99 within cluster
Cluster 4 (1 members): 0.00 within cluster
Cluster 5 (6 members): 32.27 within cluster
```

Figure 16.5 Visual Presentation (Profile Plot) of Cluster Centroids

In [15]:
```
#VB67
# Adding a new column naming each cluster (e.g., Cluster 0, Cluster 1, etc.)
centroids['cluster'] = [f'Cluster {i}' for i in centroids.index]
# Creating a parallel coordinates plot to visualize differences between clus
plt.figure(figsize=(10,5))
# Plotting each cluster's centroid profile across all variables using differ
parallel_coordinates(centroids, class_column='cluster', colormap='Dark2', li
# Place the legend outside the plot for better readability
plt.legend(loc='center left', bbox_to_anchor=(1, 0.5))
# Adjusting layout and display the final plot
plt.tight_layout()
#displaying plot
plt.show()
```

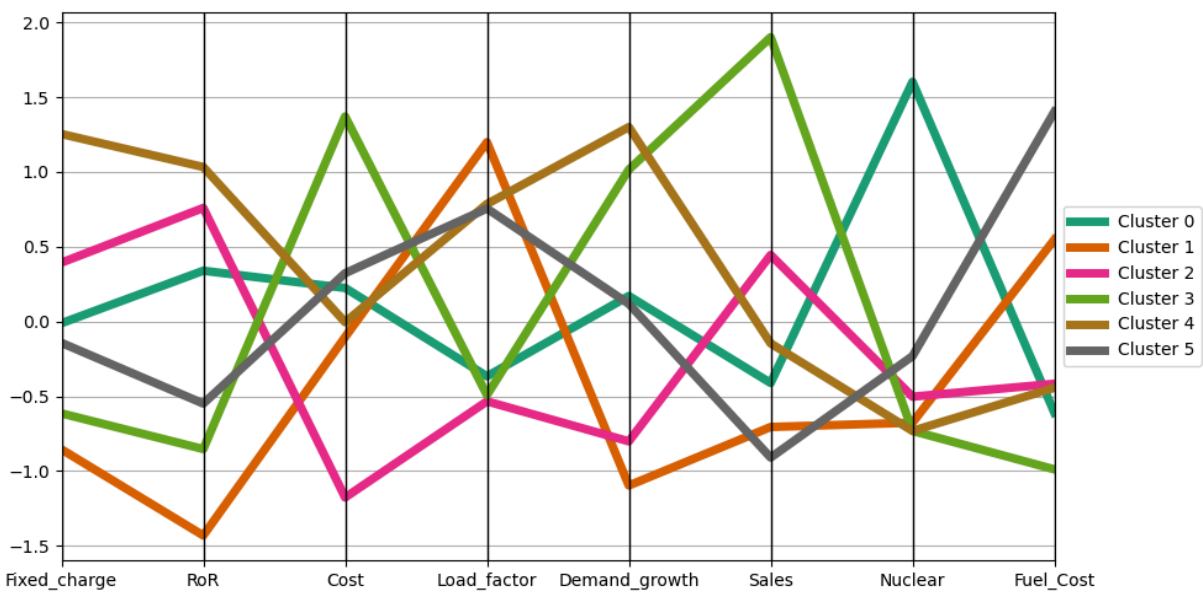


Figure 16.6 Comparing Different Choices of k in Terms of Overall Average Within-cluster Distance

When the number of clusters is not predetermined by domain requirements, we can use a graphical approach to evaluate different numbers of clusters. An “elbow chart” is a line chart depicting the decline in cluster heterogeneity as we add more clusters. Figure 16.6 shows the overall average within-cluster distance (normalized) for different choices of k. Moving from 1 to 2 tightens clusters considerably (reflected by the large reduction in within-cluster distance), and so does moving from 2 to 3 and even to 4. Adding more clusters beyond 4 brings less improvement to cluster homogeneity.

In [16]:
```
#VB67
# Initializing an empty list to store inertia (within-cluster variance) valu
inertia = []
# Running K-Means for different numbers of clusters (from 1 to 6)
for n_clusters in range(1, 7):
    kmeans = KMeans(n_clusters=n_clusters, random_state=0).fit(utilities_df_
    inertia.append(kmeans.inertia_ / n_clusters)
# Creating a DataFrame to organize the inertia values for each cluster numbe
inertias = pd.DataFrame({'n_clusters': range(1, 7), 'inertia': inertia})
# Plotting the elbow curve (number of clusters vs. average inertia)
ax = inertias.plot(x='n_clusters', y='inertia', figsize=[6, 3.5])
# Label the axes
ax.set_xlabel('Number of clusters (k)')
ax.set_ylabel('Average within-cluster squared distances')
# Set y-axis limit slightly above the maximum inertia for clarity
ax.set_ylim((0, 1.1 * inertias.inertia.max()))
# Hide legend for a cleaner look
ax.legend().set_visible(False)
# Adjusting layout and display the plot
plt.tight_layout()
#displaying plot
plt.show()
```

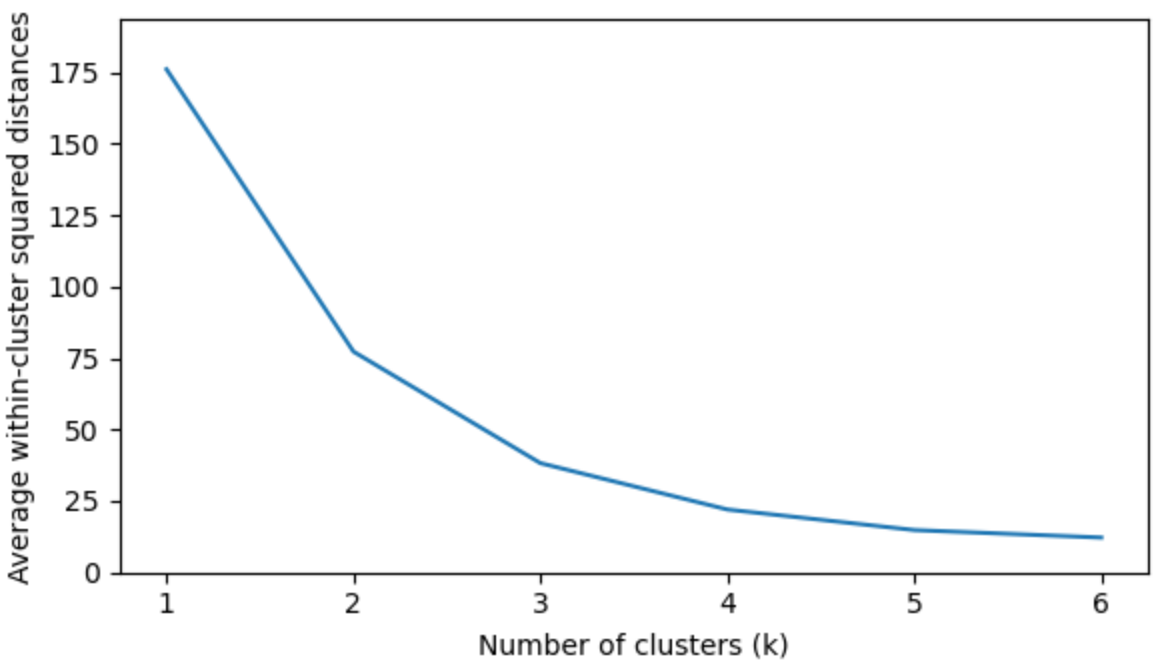


Table 16.11 Euclidean Distance Between Cluster Centroids

In [17]:
```
#VB67
# Computing and displaying the Euclidean distances between all k-means clust
pd.DataFrame(pairwise.pairwise_distances(kmeans.cluster_centers_, metric='eu
```

Out[17]:

	0	1	2	3	4	5
0	0.000	3.824	2.911	3.842	3.206	3.119
1	3.824	0.000	3.587	4.362	4.210	2.027
2	2.911	3.587	0.000	3.987	2.957	3.459
3	3.842	4.362	3.987	0.000	3.890	4.199
4	3.206	4.210	2.957	3.890	0.000	3.192
5	3.119	2.027	3.459	4.199	3.192	0.000

In [ ]: